

Code

```
import java.util.*;

public class Main
{
    public static void swap(char ch[],int i, int j){
        char temp = ch[i];
        ch[i]=ch[j];
        ch[j]=temp;
    }

    public static void permutation(char ch[], int fi){

        if (fi == ch.length){
            for(int i = 0; i< ch.length; i++){
                System.out.print(ch[i]);
            }
            System.out.println();
            return;
        }
        for(int i = fi; i < ch.length; i++){
            swap(ch, fi, i);
            permutation(ch, fi + 1);
            swap(ch, fi, i);
        }
    }

    public static void main(String[] Args){
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        char ch[] = s.toCharArray();
        permutation(ch, 0);
    }
}
```

Code (With de-duplication and dealing with repeating characters)

```
import java.util.*;

public class Main {

    public static void swap(char ch[], int i, int j) {
```

```

        char temp = ch[i];
        ch[i] = ch[j];
        ch[j] = temp;
    }

    public static void permutation(char ch[], int fi) {
        if (fi == ch.length) {
            for (int i = 0; i < ch.length; i++) {
                System.out.print(ch[i]);
            }
            System.out.println();
            return;
        }

        for (int i = fi; i < ch.length; i++) {
            // Skip duplicates: if current char same as previous AND previous wasn't used
            if (i > fi && ch[i] == ch[i - 1]) {
                continue;
            }
            swap(ch, fi, i);
            permutation(ch, fi + 1);
            swap(ch, fi, i); // backtrack
        }
    }

    public static void main(String[] Args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        char ch[] = s.toCharArray();

        // Sort first to handle duplicates properly
        Arrays.sort(ch);
        permutation(ch, 0);

        sc.close();
    }
}

```